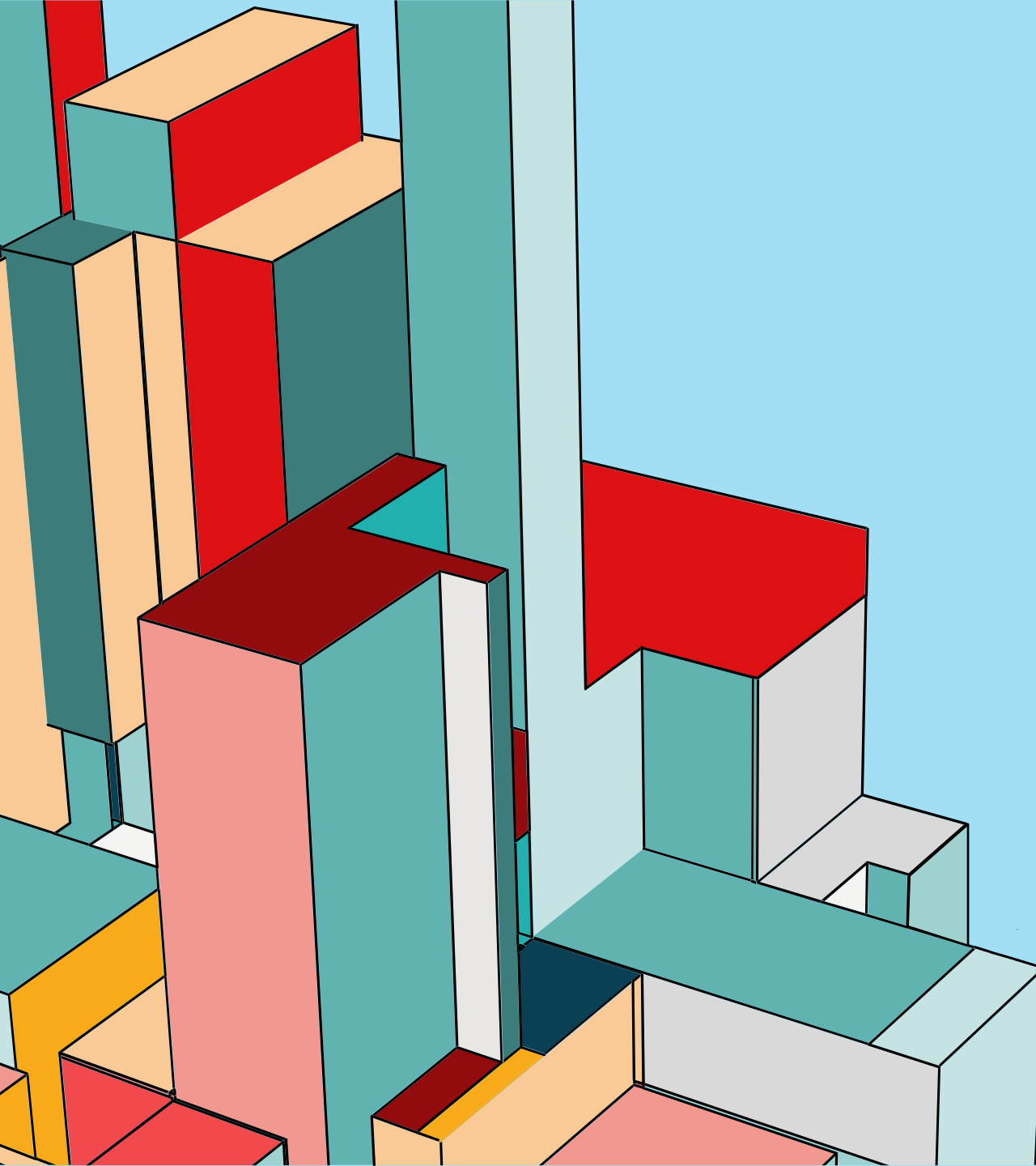


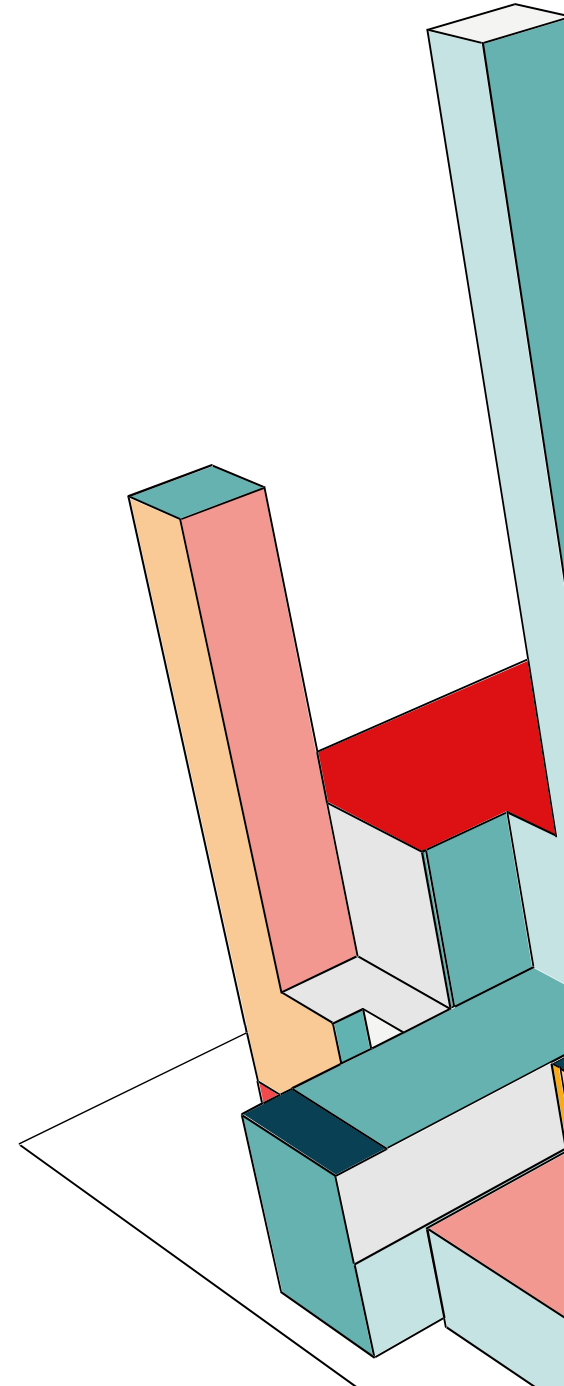


LECTURE 4 **BFS & DFS; INDUCTIVE ALGORITHMS**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

ANNOUNCEMENTS / QUESTIONS

- Office hours online today (see Discussion Forum for link)
- Responses to feedback forms
- Questions from HW?



GENERAL PROBLEM: EXPLORE A GRAPH

We have examined exhaustive search in the context of a list or array. In this case, the order of traversal is fairly obvious: Start at one end, go to the other.

When the data structure is a graph, on the other hand, we nearly always want to explore along the edges of the graph. There are a variety of orderings that might make sense.

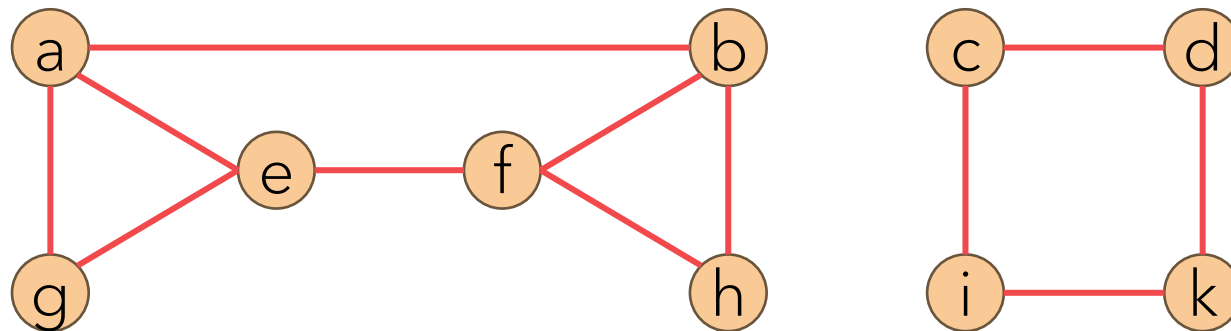
By far, though, the two most common approaches (for good reason!) are *breadth-first search* and *depth-first search*.

BREADTH-FIRST SEARCH

Core concept: Stay close to home as long as possible.

Starting at an initial vertex v , BFS will examine every node at distance 1 from v before looking at any node at distance 2, every node at distance 2 before any at distance 3, etc.

- If the graph is disconnected, we will need to use multiple starting vertices.

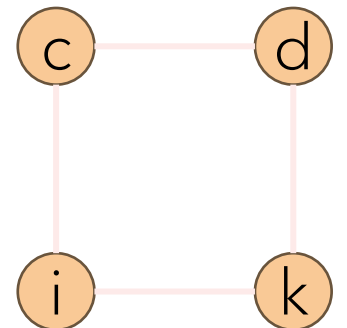
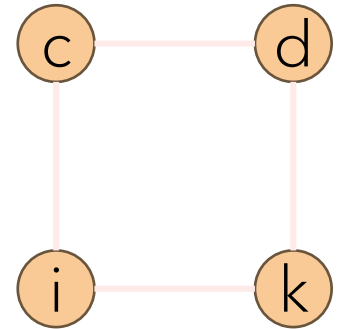
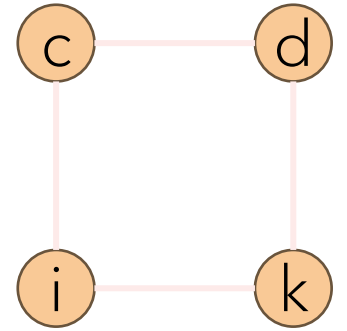
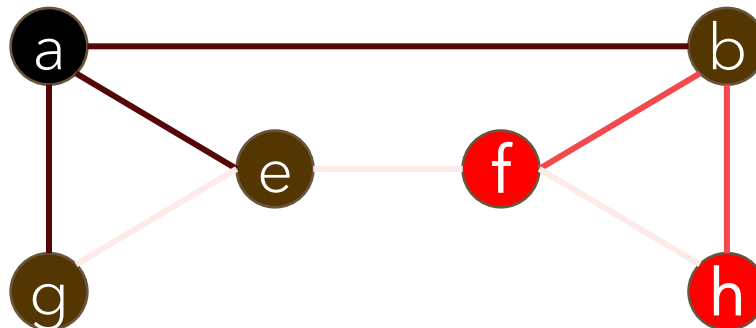
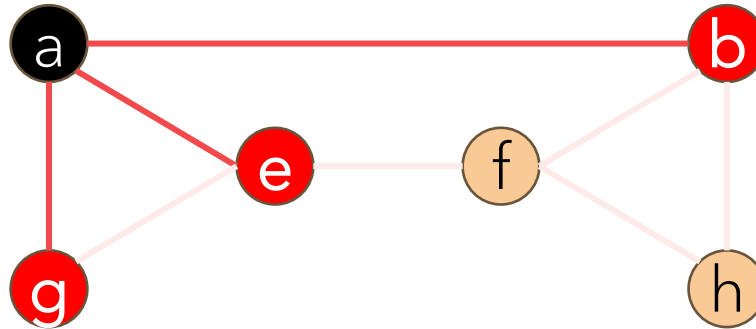
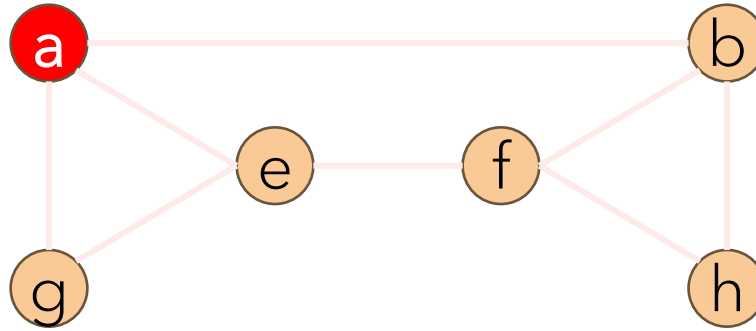


BREADTH-FIRST SEARCH

Let's say we start at node *a*.

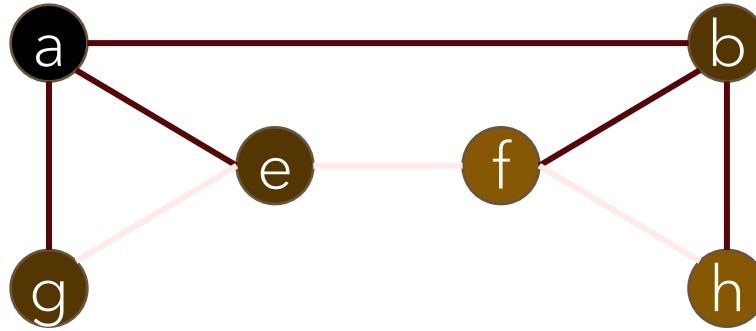
We can "see" nodes *b, e, g*.

The new nodes visible from those vertices are *f* and *h*.

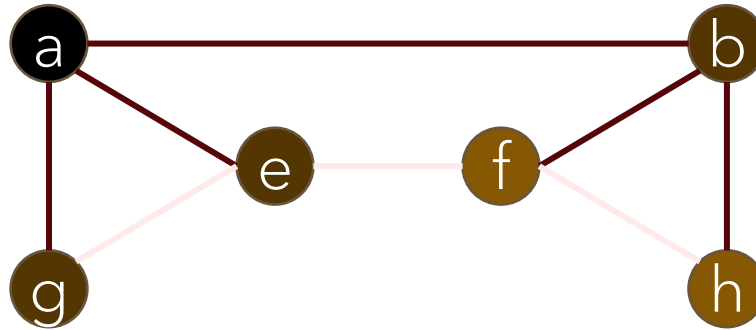


BREADTH-FIRST SEARCH

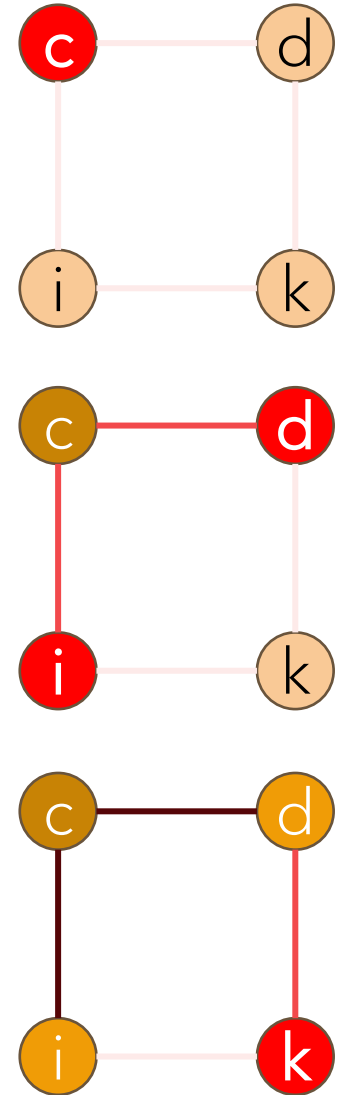
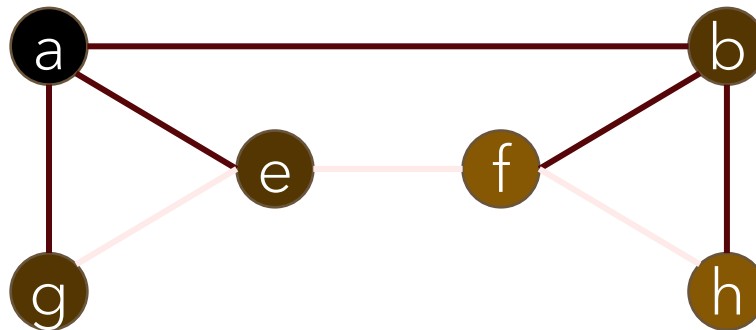
Nothing else is visible, but we're not done, so we start anew at *c*.



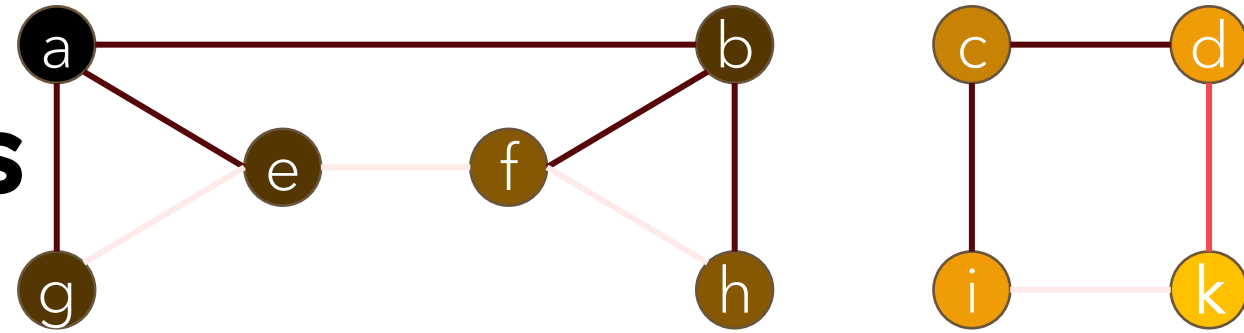
This gets us nodes *d* and *i*.



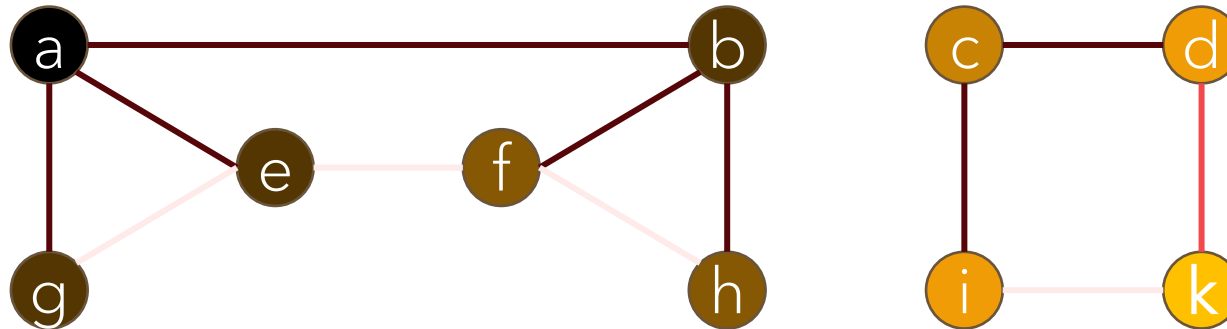
Finally, we pick up *k*.



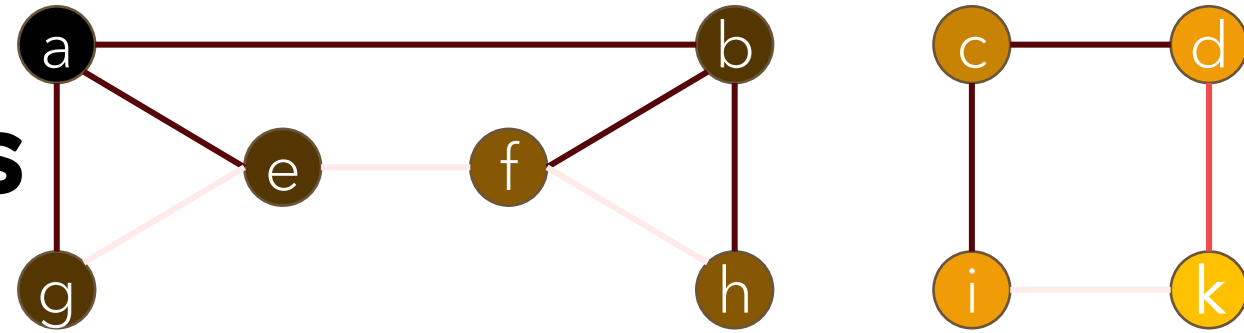
BFS TREE & CROSS EDGES



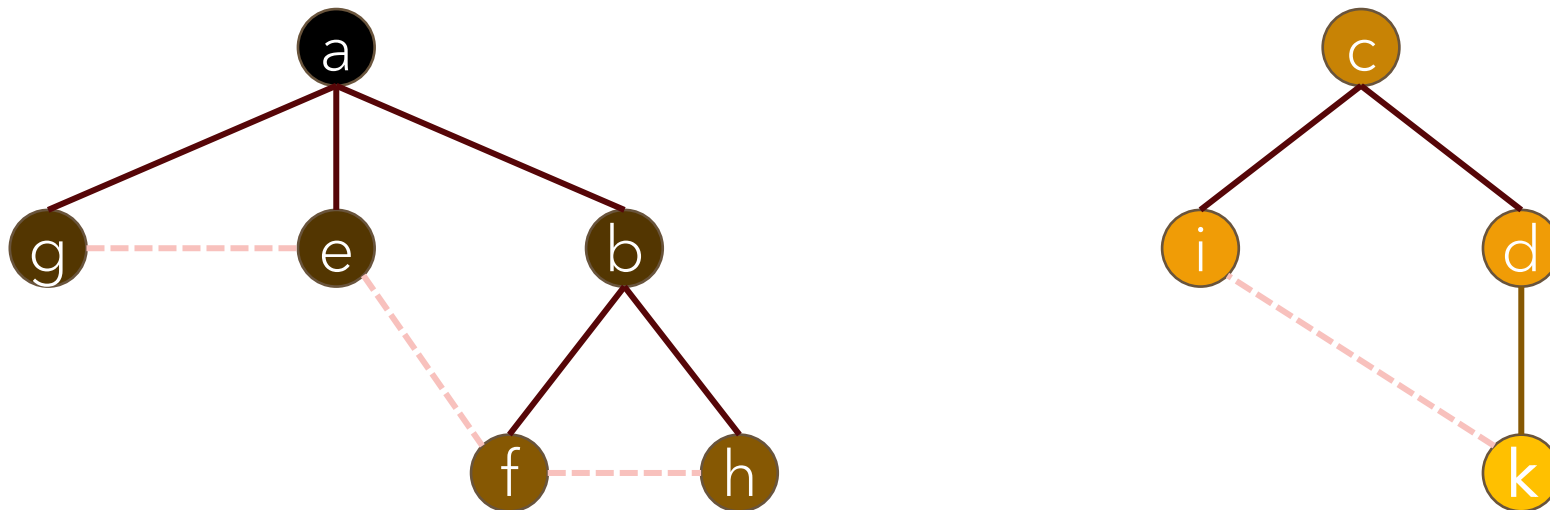
The execution of BFS induces a *forest* (a collection of trees) on the edges of the graph. The edges that BFS does not use to reach new nodes are called *cross edges*.



BFS TREE & CROSS EDGES



The execution of BFS induces a *forest* (a collection of trees) on the edges of the graph. The edges that BFS does not use to reach new nodes are called *cross edges*.



IMPLEMENTING BFS

From a practical perspective, how do we get our program to track a graph in BFS order?

Well... from our starting vertex, we want to explore all of *its* neighbors before we explore any of *their* neighbors.

We should explore first the nodes that we see first.

Put another way - we should explore nodes in a first-in, first-out order, which points us to using a queue.

BFS PSEUDOCODE

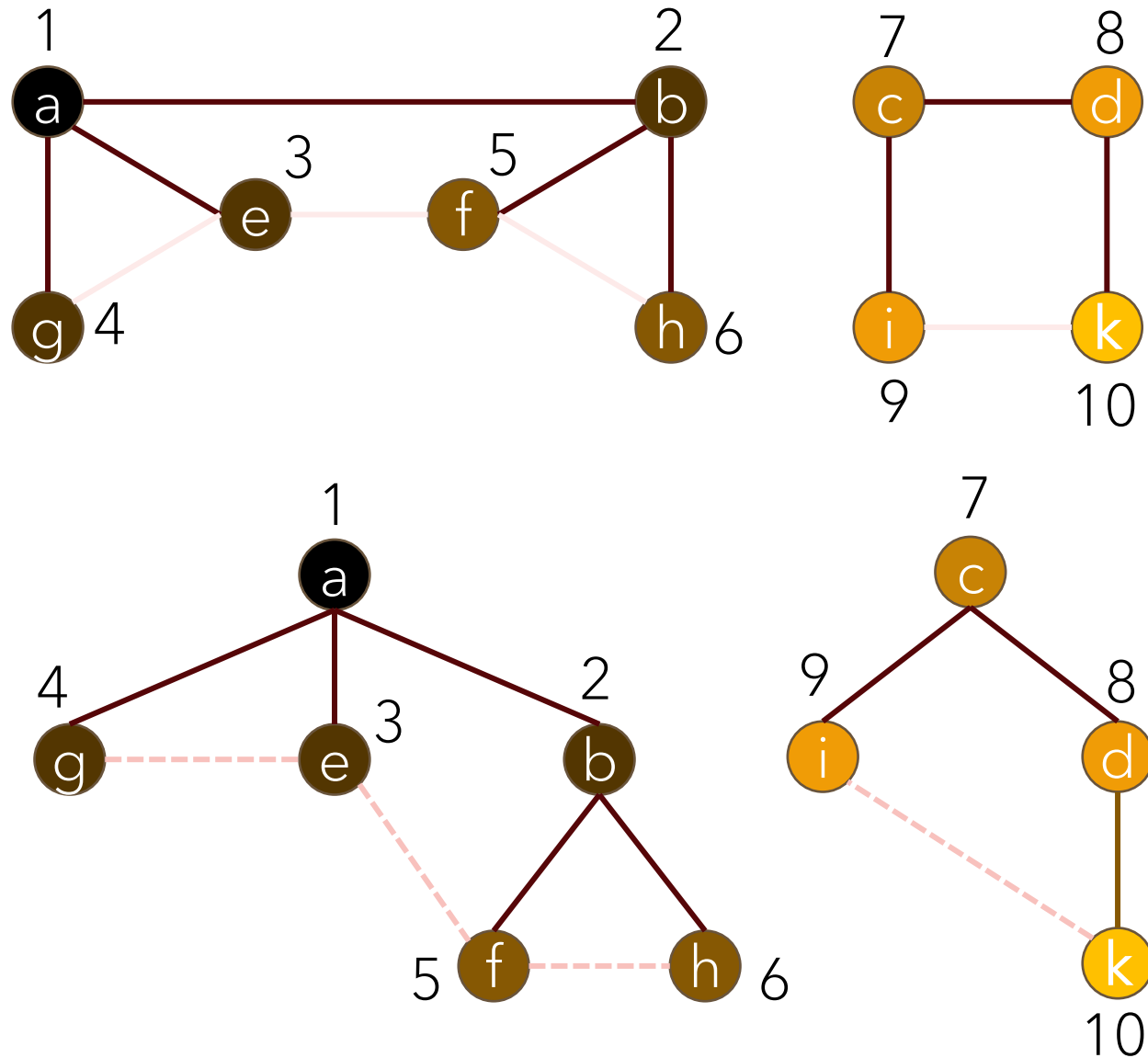
Note: We assume here G is connected; otherwise, we'll need
a wrapper function to call this on each connected component

```
def bfs( $G, v_1$ ):  
     $t = 1$ ; set  $\text{time}(v) = 0$  for all vertices  $v$  in  $G$   
     $\text{time}(v_1) = t$ ;  $\text{parent}(v_1) = \text{NULL}$ ; add  $v_1$  to the queue  
    while the queue is not empty:  
         $v =$  next vertex in queue (& remove from queue)  
        for each neighbor  $n$  of  $v$ :  
            if  $\text{time}(n) == 0$ :  
                 $t = t + 1$   
                 $\text{time}(n) = t$ ;  $\text{parent}(n) = v$ ; add  $n$  to the queue
```

BFS TIMES & TREES

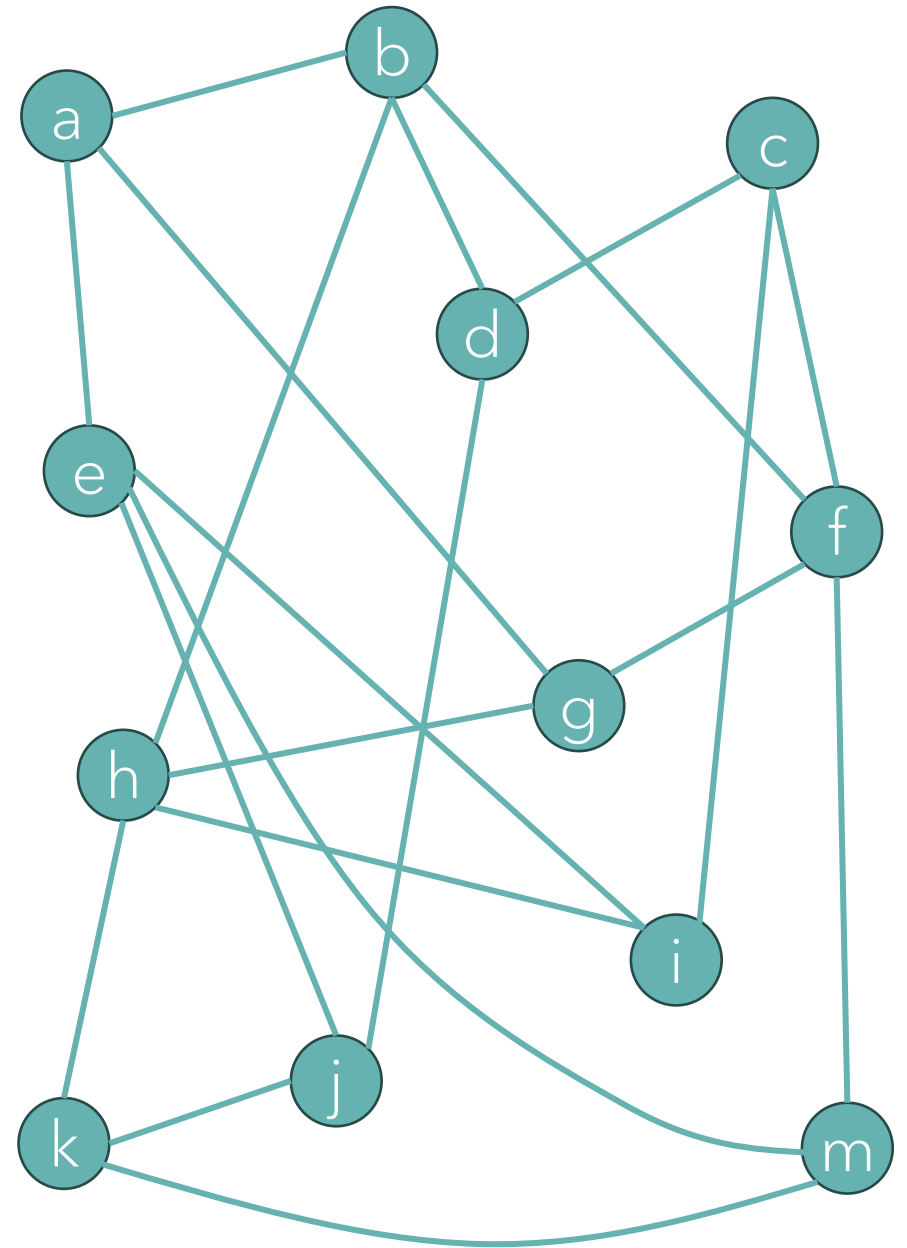
The “times” in the given algorithm explicitly tell us the order the nodes are visited.

Observe that the cross edges are always between different branches of the BFS – you can never have a cross edge from a node to its ancestor if the graph is undirected. (Why not?)



QUICK CHECK-IN

Starting from *a*, and with ties broken alphabetically, which node will be explored last by breadth-first search?



SUBMITTY POLL #1

Which node is explored last?

Enter a single letter.



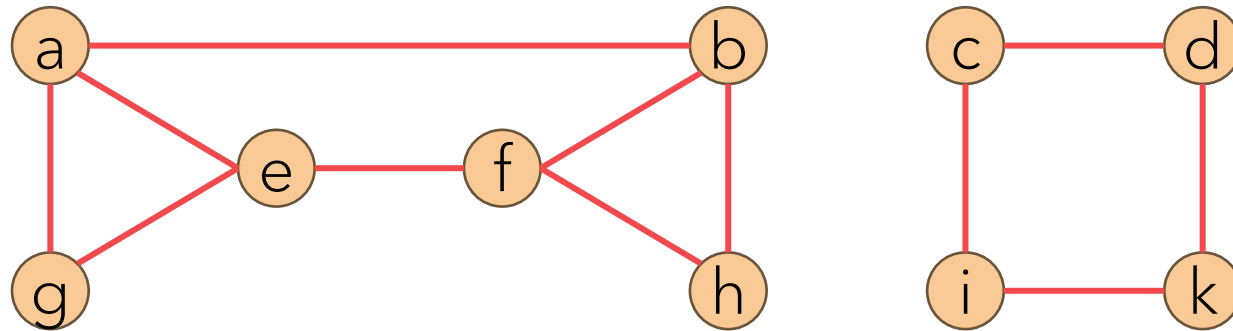
<https://submittity.cs.rpi.edu/courses/s26/csci2300/polls/7>

DEPTH-FIRST SEARCH

Core concept: Finish what you started.

Starting at an initial vertex v , DFS will pick a path and follow it as far as it can before trying any alternative paths.

- The classic maze approach of “put your left hand on the wall and just keep walking” is essentially a DFS of the maze, where each intersection is a vertex.

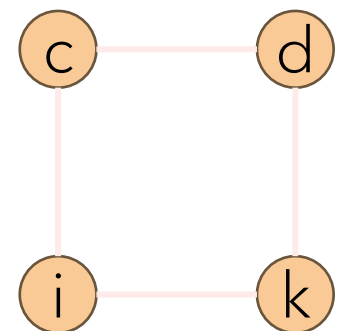
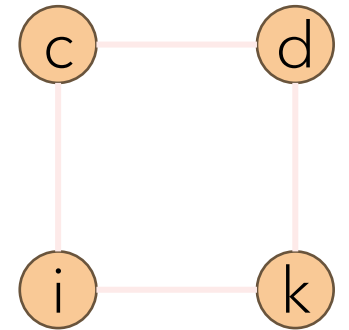
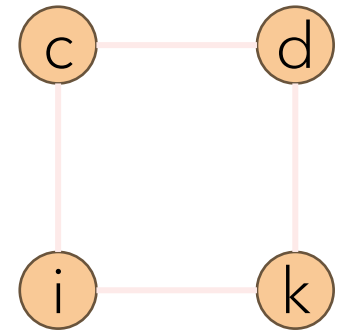
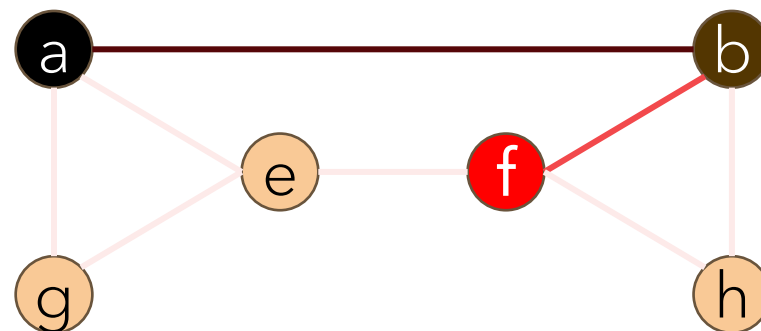
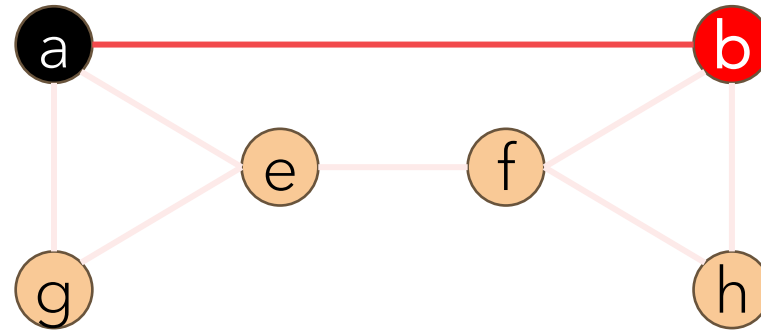
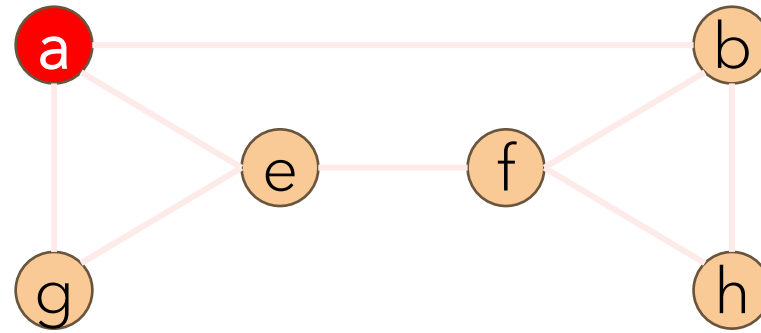


DEPTH-FIRST SEARCH

Once again, we start at node *a*.

We'll go to *b* next...

... and then on to *f*.

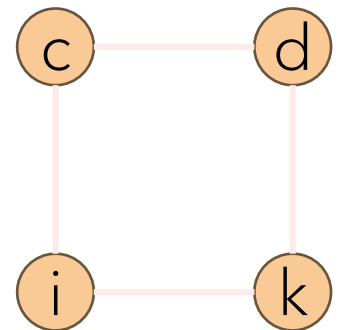
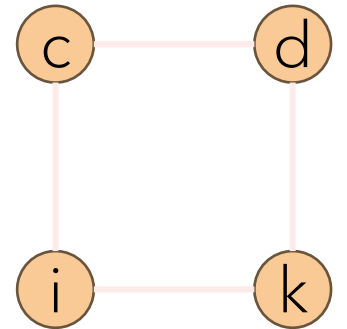
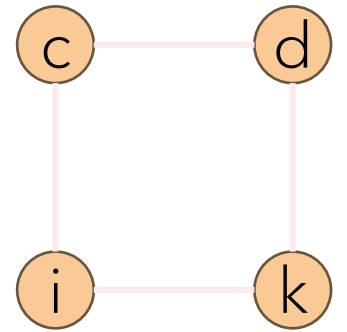
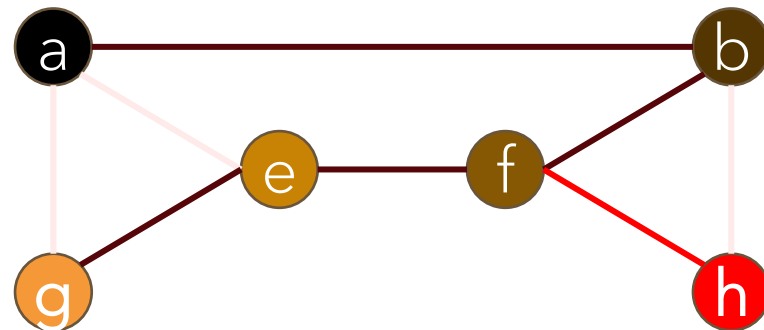
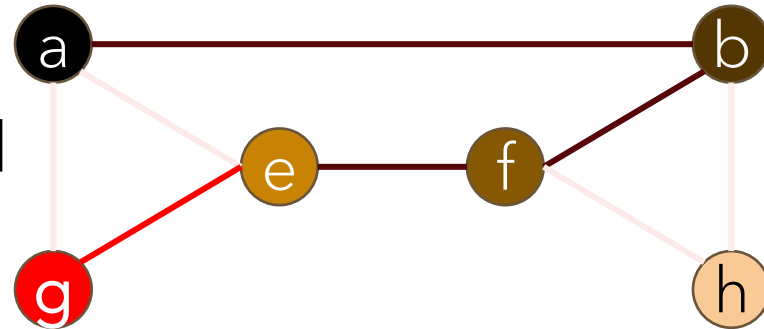
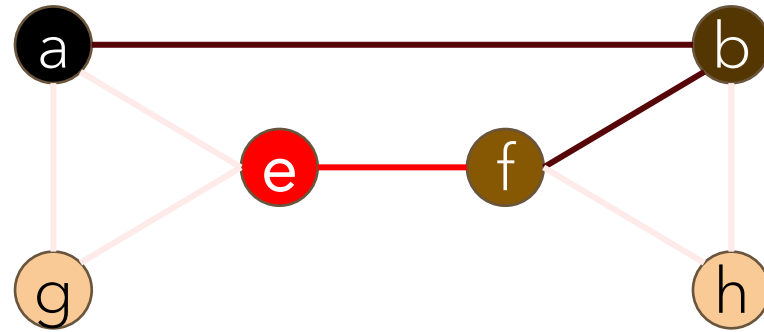


DEPTH-FIRST SEARCH

Then to *e*.

... and *g* finally gets us to a dead end, and we start backtracking.

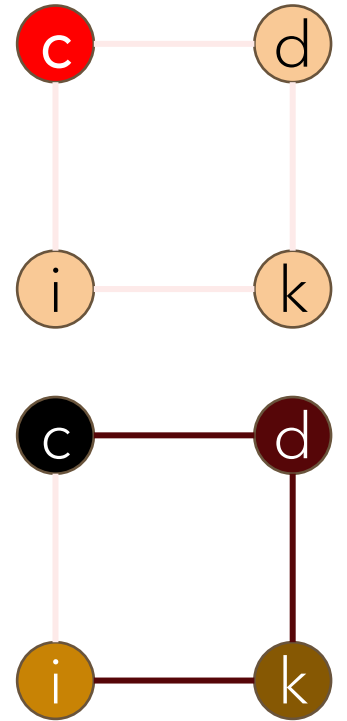
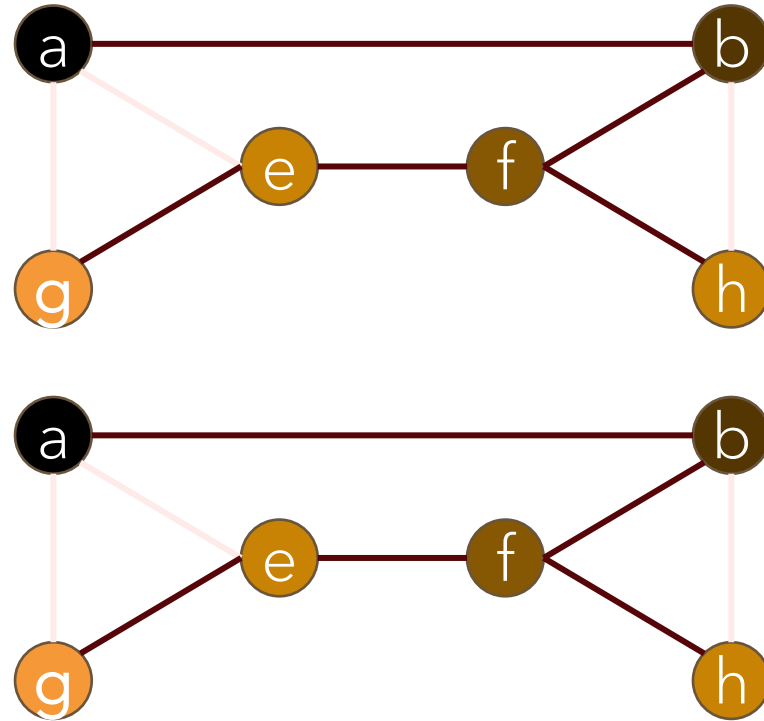
e has no new neighbors, but *f* does. Add *h*.



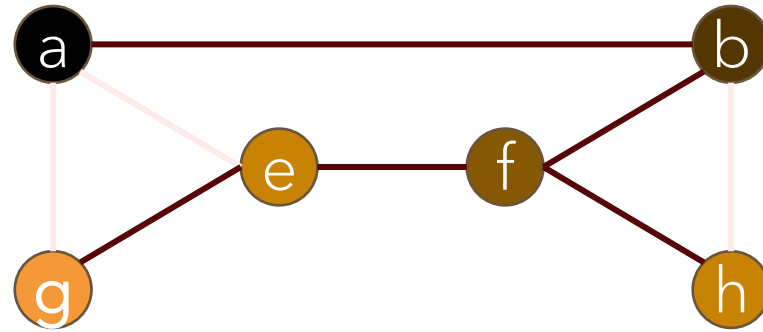
DEPTH-FIRST SEARCH

Again, we'll need to restart at *c*.

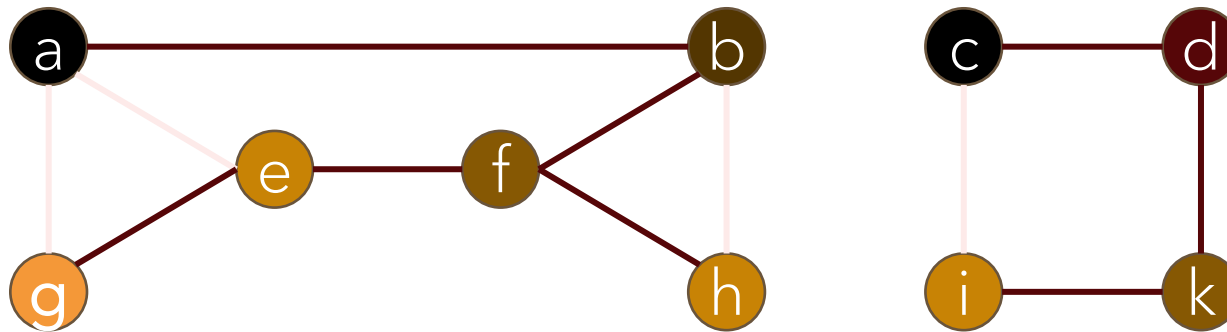
From *c*, we just follow the loop around: *d* then *k* then *i*.



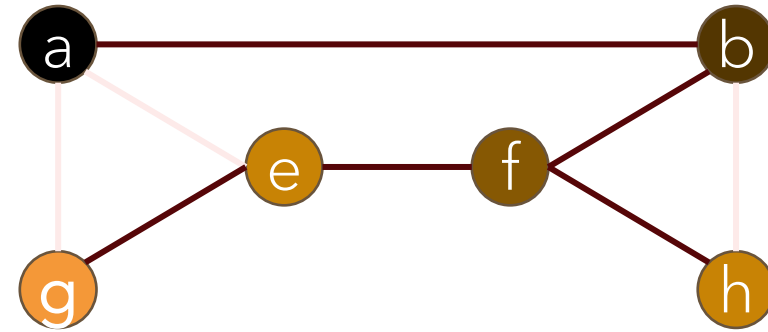
DFS TREE & BACK EDGES



DFS also induces a forest on its graph.
This time (for reasons that will be obvious in a moment) the non-tree edges are called *back edges*.

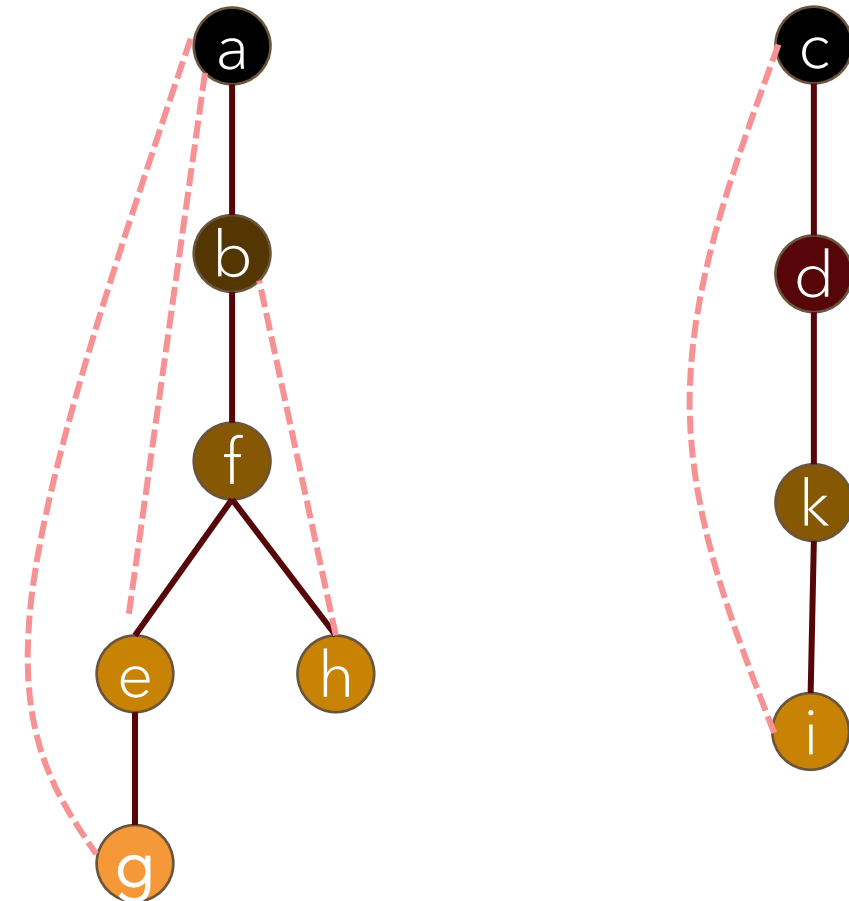


DFS TREE & BACK EDGES



DFS also induces a forest on its graph. This time (for reasons that will be obvious in a moment) the non-tree edges are called *back edges*.

Observe that every back edge connects a node to an ancestor; back edges on an undirected graph never connect to other branches of the DFS tree. (Why not?)



IMPLEMENTING DFS

Same question as before: How do we get our program to track a graph in DFS order?

If we were talking about queues before, you might expect that we should be talking about stacks now – and you'd be right!

This makes intuitive sense: The first vertices that we see are actually the last ones that we explore. Each time we “open” a new node, we make sure we completely finish that node before finishing up previous ones.

We don't need to explicitly declare a stack if we use a recursive implementation...

DFS PSEUDOCODE

Same note about connected graphs applies here

```
def DFS( $G, v_1$ ):
```

```
    for all  $v \in G$ , set  $\text{in}(v) = 0$  and  $\text{out}(v) = 0$ 
```

```
     $t = \text{rec\_dfs}(G, v_1, \text{NULL}, 0)$ 
```

```
def rec_dfs( $G, v, p, t$ ):
```

```
     $t = t + 1$ 
```

```
     $\text{in}(v) = t$ ;  $\text{parent}(v) = p$ 
```

```
    for each neighbor  $n$  of  $v$ :
```

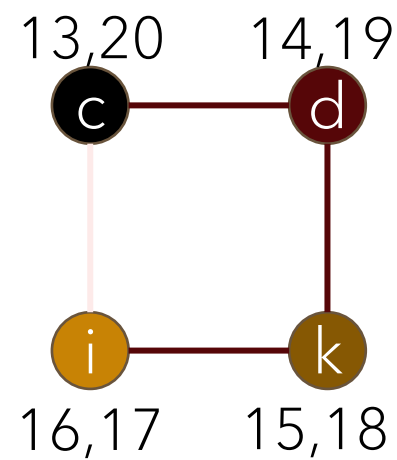
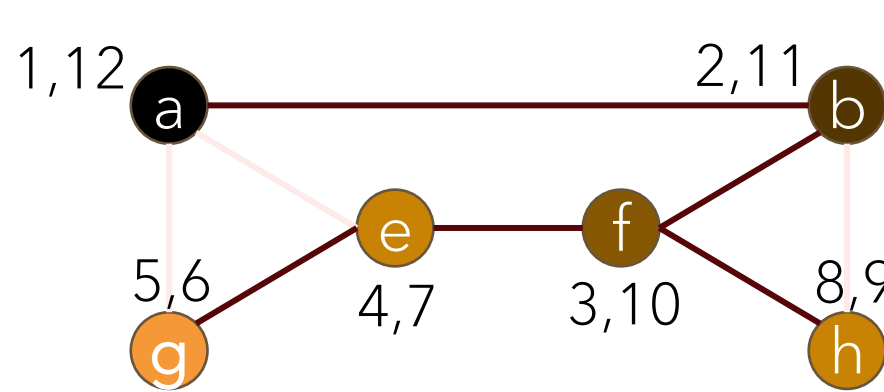
```
        if  $\text{in}(n) == 0$ :
```

```
             $t = \text{rec\_dfs}(G, n, v, t)$ 
```

```
     $\text{out}(v) = t + 1$ 
```

```
    return  $t + 1$ 
```

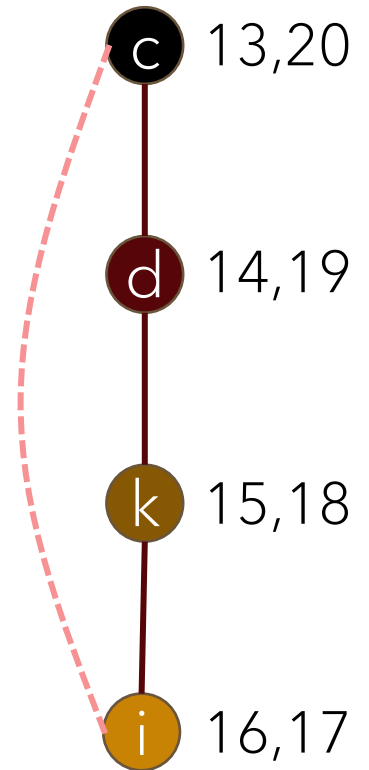
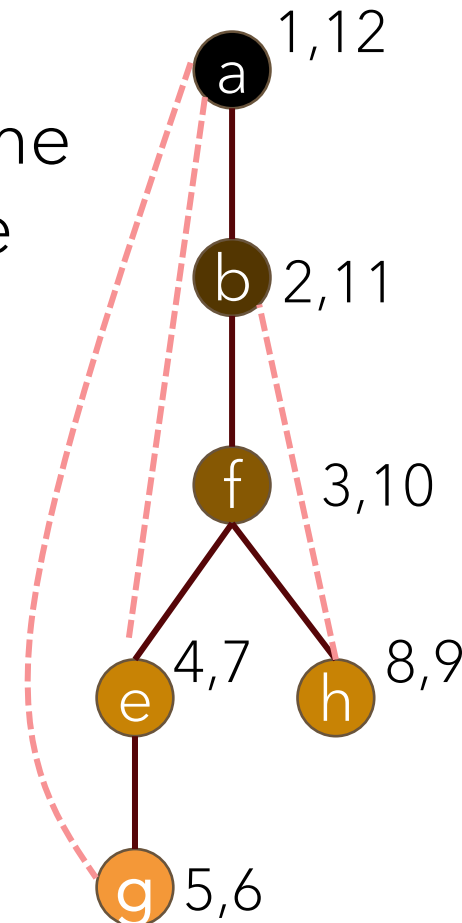
DFS TIMES



Observe that DFS gives us two times for each node in the graph: the time we begin exploring it ("**in**") and the time we finish exploring it ("**out**").

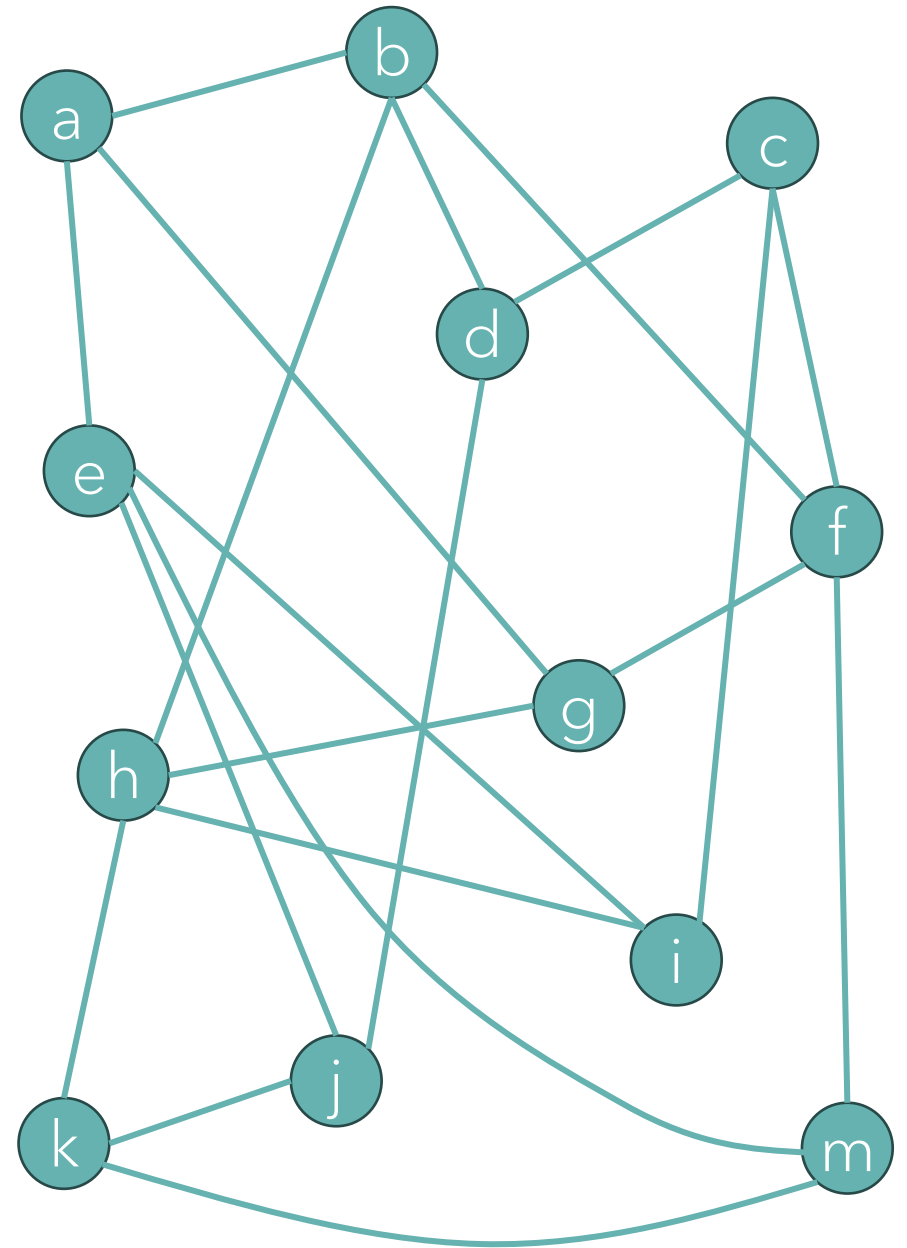
This gives us two distinct ways to order the vertices.

- "**in**": a,b,f,e,g,h,c,d,k,i
- "**out**": g,e,h,f,b,a,i,k,d,c



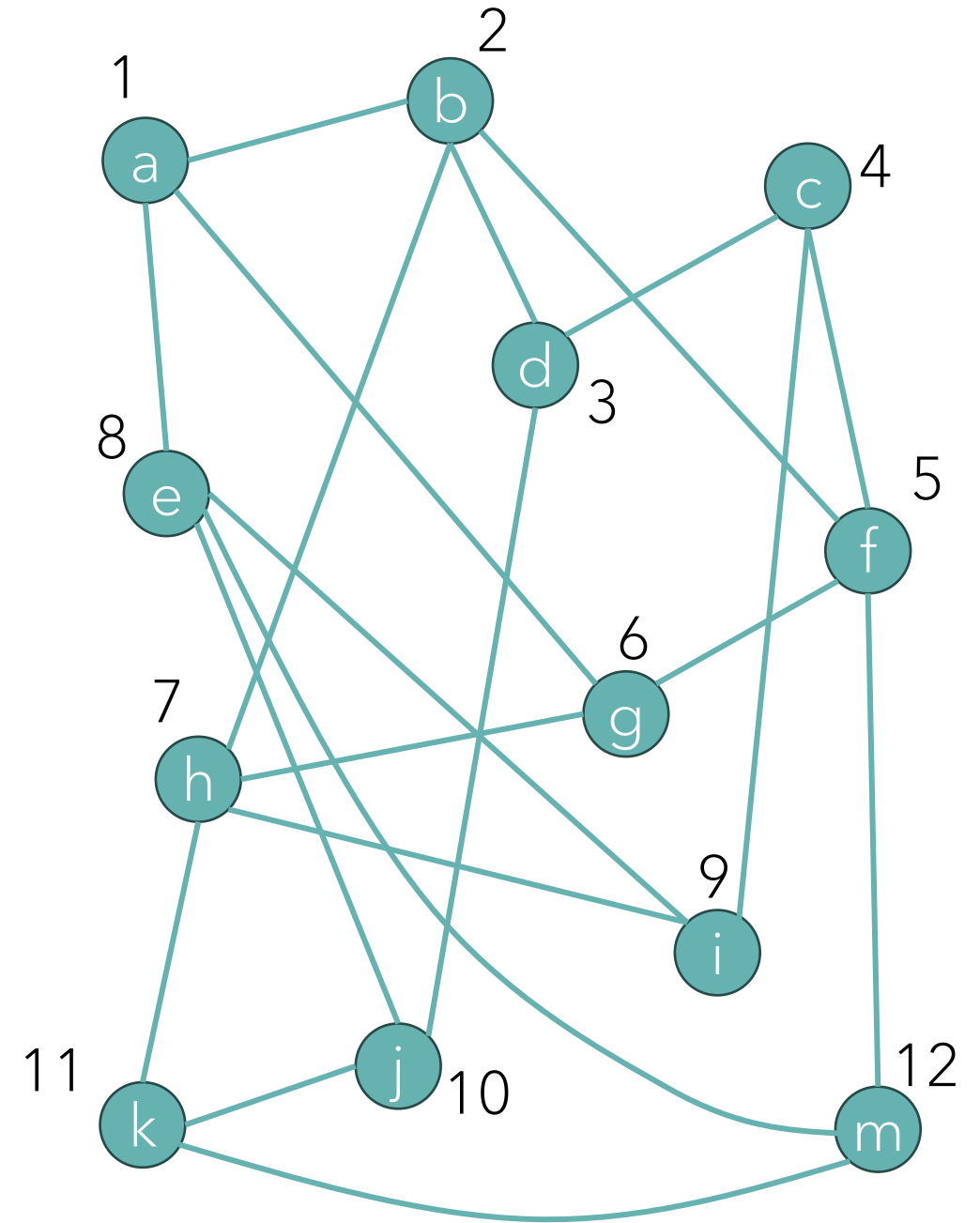
QUICK CHECK-IN

Starting from *a*, and with ties broken alphabetically, which node will be arrived at last ("in" time) by depth-first search?



QUICK CHECK-IN

Starting from *a*, and with ties broken alphabetically, which node will be arrived at last ("in" time) by depth-first search?



IMPORTANCE OF BFS AND DFS

Immense. These two algorithms get used in many, many places, even when it's not obviously a graphs application.

For graphs, both of them can be used to check a graph for connectivity and to see if there are any cycles present.

DFS can also be used to find *cut vertices* (nodes whose removal disconnects the graph). BFS can be used to find paths that take the fewest edges.

Both algorithms are useable on directed graphs; the non-tree edges then convey some more information, but we're not going to dive into that any further today.

BFS RUNTIME ANALYSIS

```
def bfs( $G, v_1$ ):  
     $t = 1$ ; set  $\text{time}(v) = 0$  for all vertices  $v$  in  $G$   
     $\text{time}(v_1) = t$ ;  $\text{parent}(v_1) = \text{NULL}$ ; add  $v_1$  to the queue  
    while the queue is not empty:  
         $v =$  next vertex in queue (& remove from queue)  
        for each neighbor  $n$  of  $v$ :  
            if  $\text{time}(n) == 0$ :  Critical operation  
                 $t = t + 1$   
                 $\text{time}(n) = t$ ;  $\text{parent}(n) = v$ ; add  $n$  to the queue
```

What's the runtime?

BFS RUNTIME ANALYSIS

```
def bfs( $G, v_1$ ):  
     $t = 1$ ; set  $\text{time}(v) = 0$  for all vertices  $v$  in  $G$   
     $\text{time}(v_1) = t$ ;  $\text{parent}(v_1) = \text{NULL}$ ; add  $v_1$  to the queue  
    while the queue is not empty:  
         $v =$  next vertex in queue (& remove from queue)  
        for each neighbor  $n$  of  $v$ :  
            if  $\text{time}(n) == 0$ :  Critical operation (usually)  
                 $t = t + 1$   
                 $\text{time}(n) = t$ ;  $\text{parent}(n) = v$ ; add  $n$  to the queue
```

The **for** loop executes $2E$ times, where E is the number of edges.

In very sparse graphs, there could be more vertices than edges.

So to be fully correct, we write the runtime as $\Theta(V + E)$.

DFS RUNTIME ANALYSIS

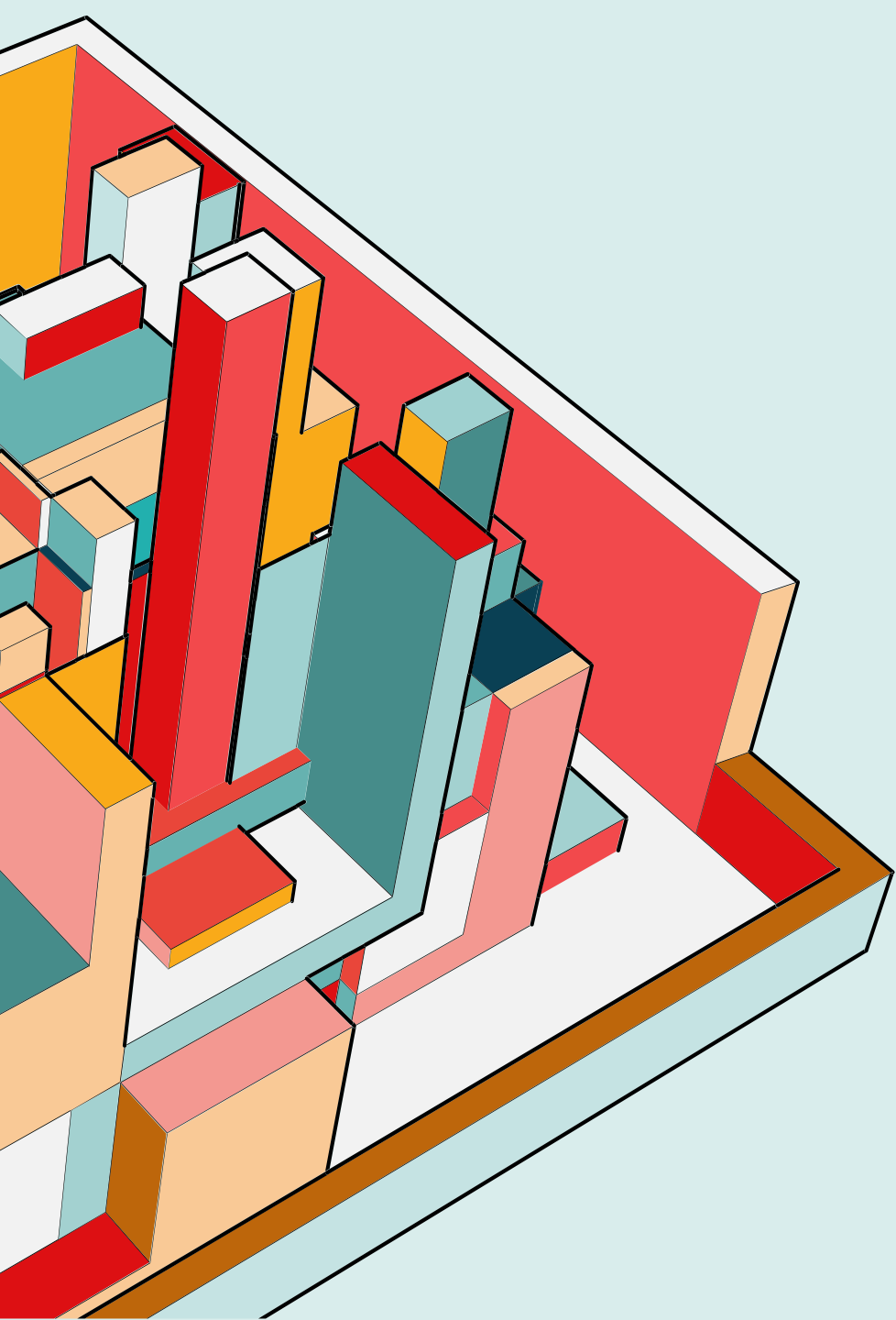
```
def DFS( $G, v_1$ ):  
    for all  $v \in G$ , set  $\text{in}(v) = \emptyset$  and  $\text{out}(v) = \emptyset$   
     $t = \text{rec\_dfs}(G, v_1, \text{NULL}, \emptyset)$ 
```

```
def rec_dfs( $G, v, p, t$ ):  
     $t = t + 1$   
     $\text{in}(v) = t$ ;  $\text{parent}(v) = p$   
    for each neighbor  $n$  of  $v$ :  
        if  $\text{in}(n) == \emptyset$ :  
             $t = \text{rec\_dfs}(G, n, v, t)$   
     $\text{out}(v) = t + 1$   
    return  $t + 1$ 
```

Same question: What's the runtime?

It's challenging to write a recurrence relation here, but we can effectively reuse the reasoning from BFS: We are going to look at each vertex once and each edge twice.

Thus, the runtime is $\Theta(V + E)$.



INDUCTIVE ALGORITHMS

WHAT IS AN “INDUCTIVE ALGORITHM”?

In this case, that’s not a term you’re likely to see elsewhere, precisely because it’s too broad – many (most?) algorithms (including several categories that we’ll hit later on) rely on inductive thinking.

I’m using it here, though, because I want to cue you to think about how inductive proofs work:

- Assume that it works for a problem of a certain size.
- Given that, show how to make it work for a problem of the next size up.

WHAT IS THE “NEXT SIZE UP”?

Just like inductive proofs, this is allowed to vary:

It could be:

- $n + 1$ (or $n + c$, for some constant c)
- $2n$ (or cn , for some constant c)
- A variable amount each time! (essentially strong induction)

EXAMPLES OF EACH TYPE

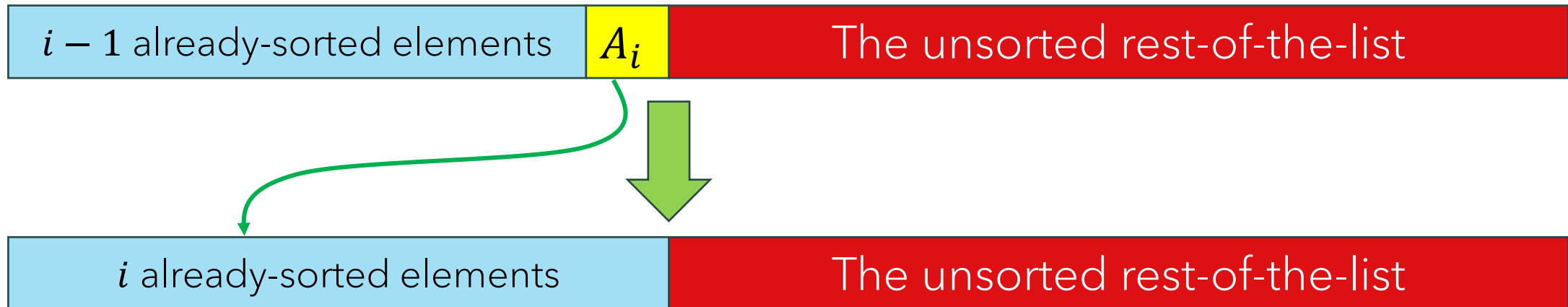
- $n + 1$ (or $n + c$, for some constant c)
 - Go back to our brute-force algorithm for n^k . We could get to the same place by taking a “top-down” view: $n^k = n \cdot n^{k-1}$
- $2n$ (or cn , for some constant c)
 - The repeated squaring approach to n^k can fit this description.

$$n^k = \begin{cases} n^{k/2} \cdot n^{k/2} & \text{when } k \text{ is even} \\ n \cdot n^{(k-1)/2} \cdot n^{(k-1)/2} & \text{when } k \text{ is odd} \end{cases}$$

- A variable amount each time
 - Euclid’s GCD algorithm.

INSERTION SORT

- The best of the quadratic sorts is usually held to be *insertion sort*. The core idea of insertion sort is to take each item as you encounter it and place it into the already-sorted portion of the list.



INSERTION SORT VISUALIZATIONS

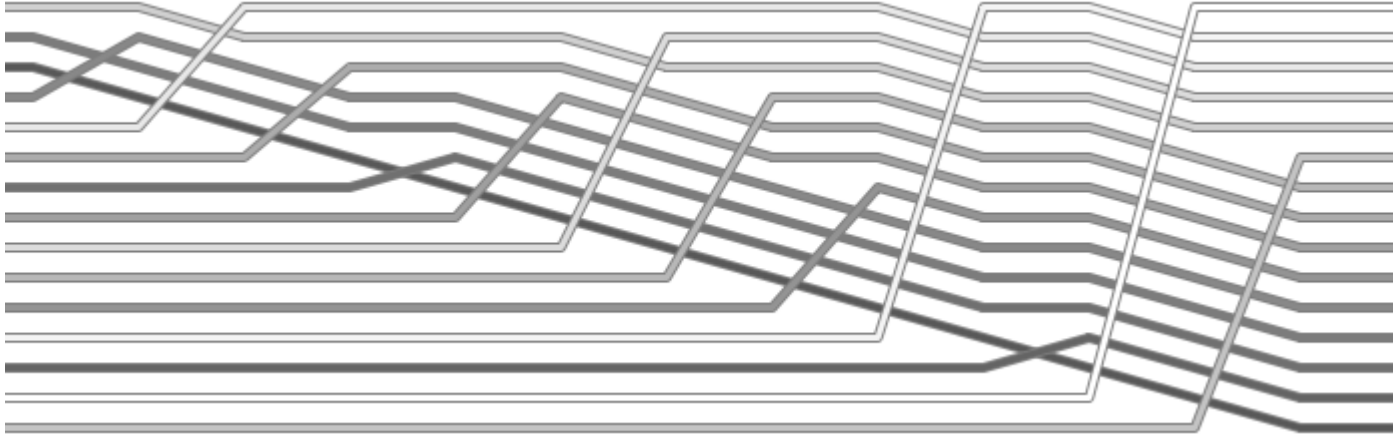
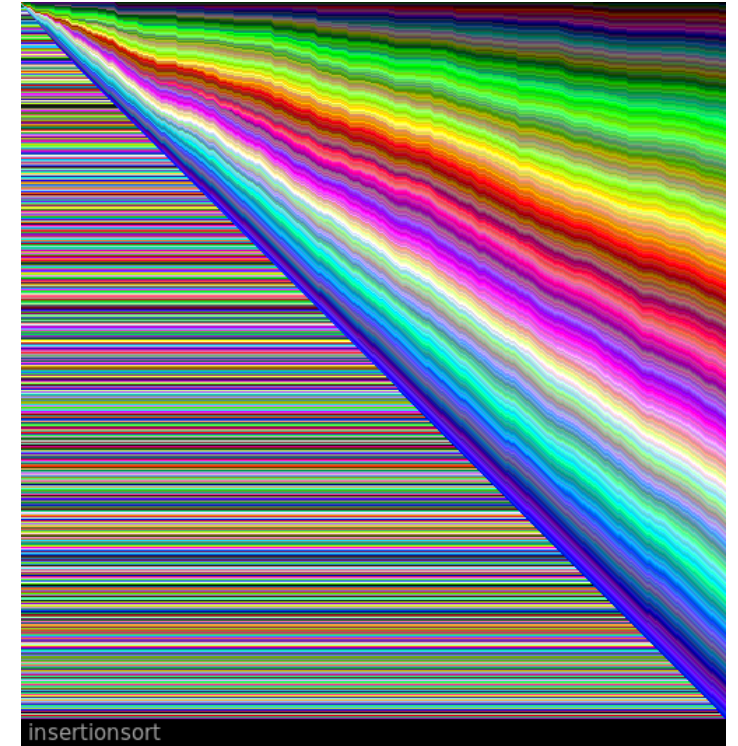


Image credit (both images): Aldo Cortesi, <https://corte.si>



<https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html>

INSERTION SORT, RECURSIVELY

```
def insSort(A,n):  
    if n == 1:  # Look! A base case!  
        return  
    insSort(A,n-1)  
    Slide A[n-1] down into place
```

In practice, this tends to run faster when we write it in a bottom-up, non-recursive fashion.

INSERTION SORT, NON-RECURSIVELY

Note: Actual code

```
def insSort(A,n):
```

```
    for i = 1 to n-1:    # skip 0; size 1 list is sorted
```

```
        v = A[i]
```

```
        j = i-1
```

```
        while (j >= 0) && (A[j] > v):
```

```
            A[j+1] = A[j]
```

```
            j = j - 1
```

```
        A[j+1] = v
```



Critical operation

RUNTIME ANALYSIS OF INSERTION SORT

How many times will we need to compare two list elements?

Side note: Comparing list elements will almost certainly be slower than checking if our index is still ≥ 0 .

Best case: The list is sorted. Each time we hit the **while** loop, we fail immediately. We hit the loop once per element, so this gives us $\Theta(n)$ comparisons.

Worst case: The list is reverse-sorted. The **while** loop performs i iterations each time we hit it. $\Theta(n^2)$

Average case: Requires some knowledge of how the list is likely to be arranged. Uniformly random gives $\Theta(n^2)$.

ADVANTAGES OF INSERTION SORT

Adaptive: When the data is almost-sorted, performance is linear.

Online: Can sort as elements arrive; doesn't need the full list to begin its work

Stable: If two or more elements have the same value, their relative order will be maintained.

In-place: Requires only $O(1)$ memory beyond the list itself

Simple: Very easy to implement

Fast on small cases: Depending on architecture and implementation, can be faster than merge on sizes $< \sim 50$

SUBMITTY POLL #2

Which of the listed advantages apply to selection sort?

Adaptive, online, stable, in-place,
simple, fast on small cases



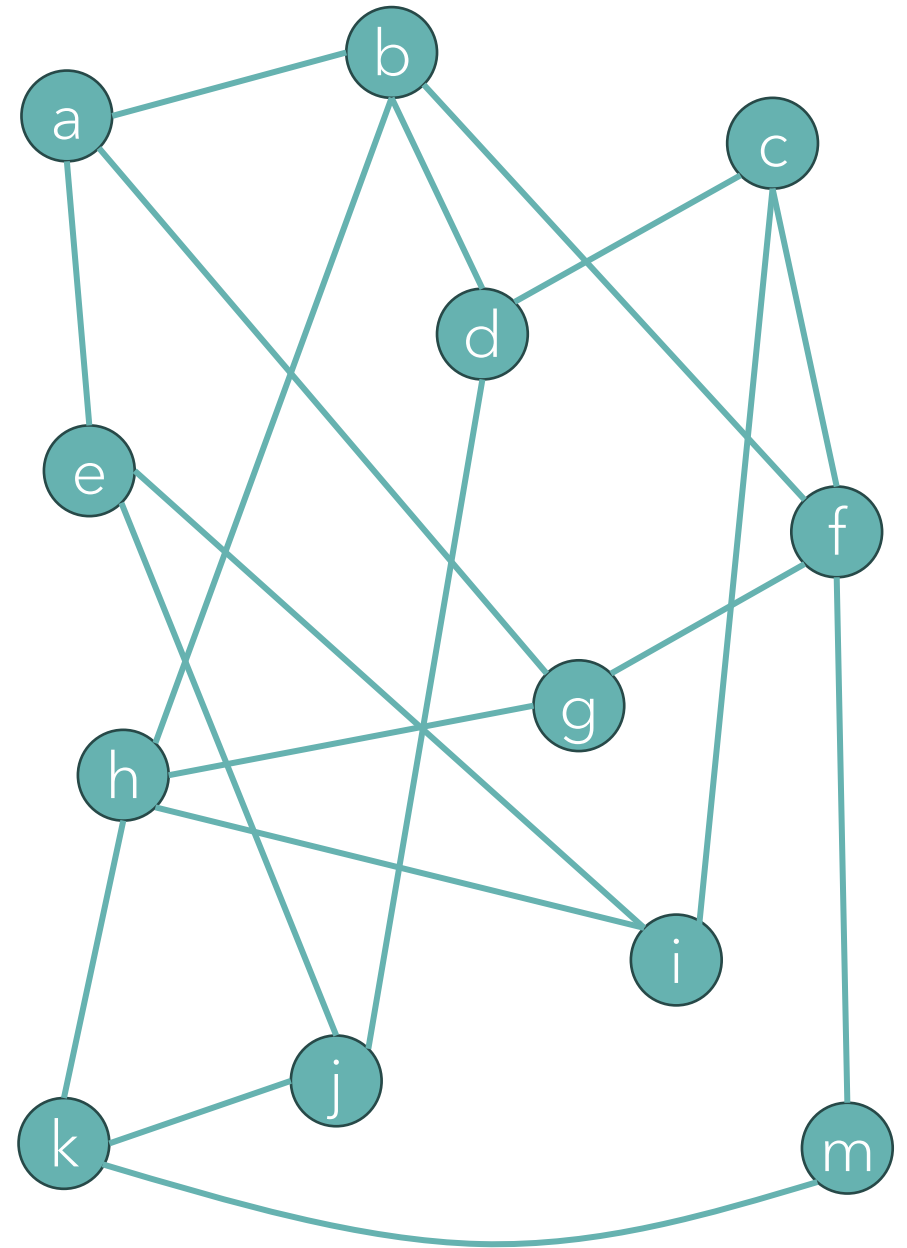
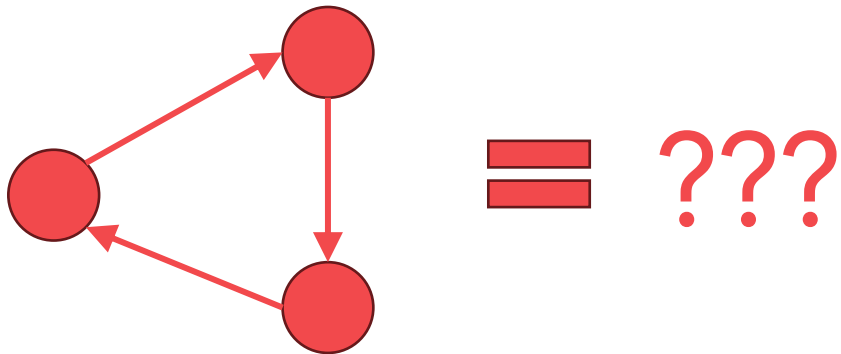
<https://submittity.cs.rpi.edu/courses/s26/csci2300/polls/6>

TOPOLOGICAL SORT

Q: How do you sort the vertices of a graph?

Nothing about an undirected graph provides an intrinsic notion of order.

But a directed graph may give us at least a partial ordering, as long as there are no cycles.

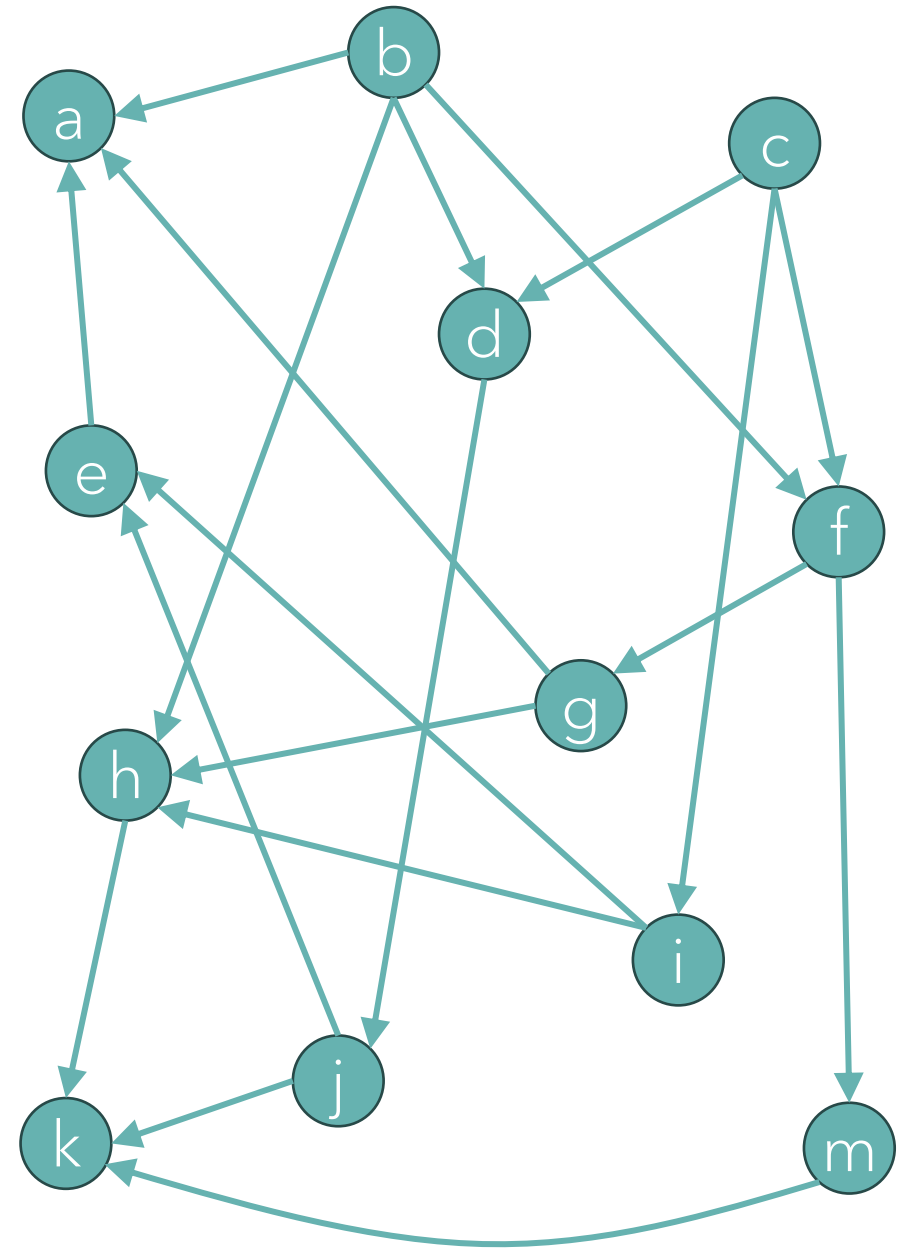


TOPOLOGICAL SORT

A *directed acyclic graph* or *DAG* is an important structure in computer science and graph theory.

Among other things, it provides a *partial order* of the vertices: We can say that if there is a path $x \rightarrow y$, then x comes before y .

It is only a partial order because if there is no path from x to y nor from y to x , we can put them in either order.



TOPOLOGICAL SORT

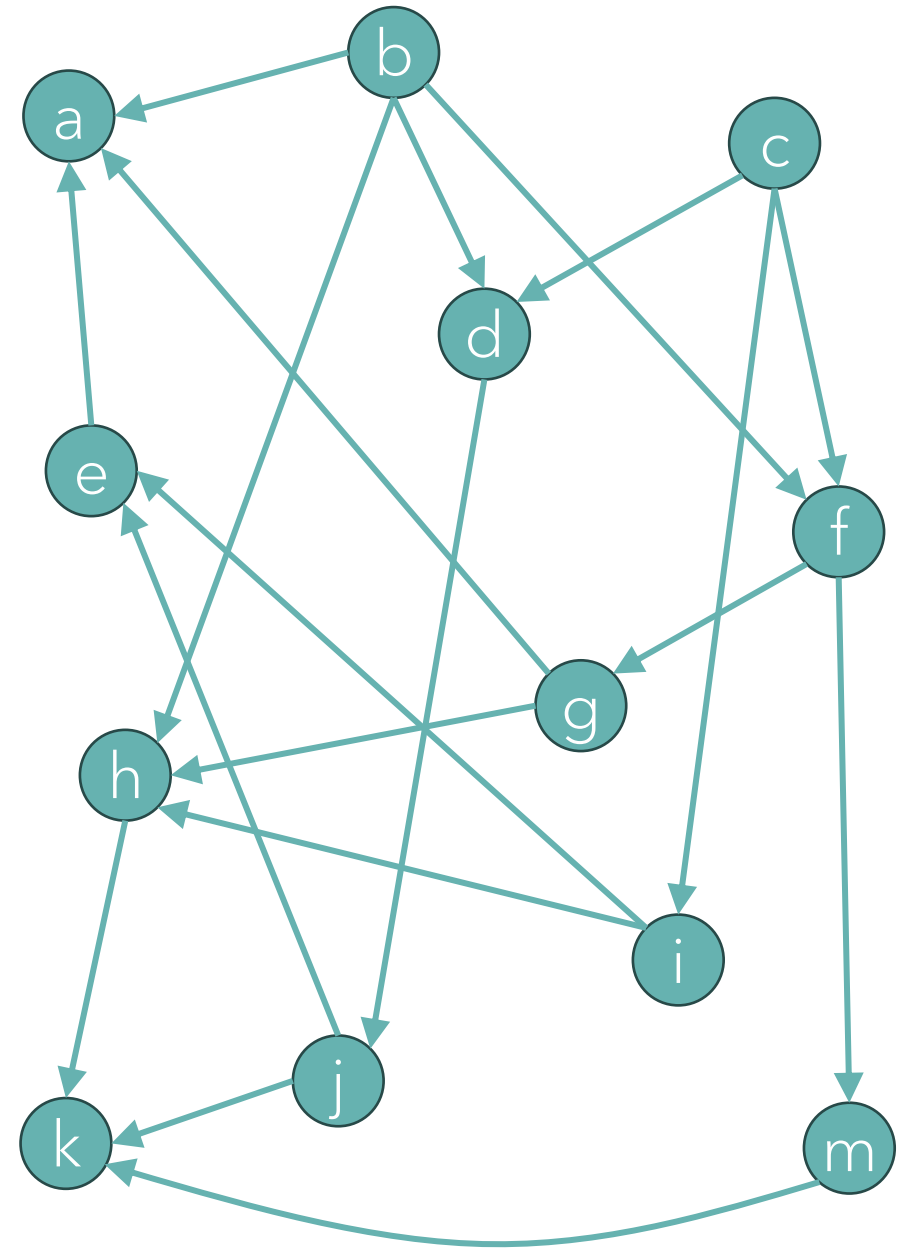
A *topological sort* is then just the process of putting the vertices in an order such that, if x comes after y , then there is no directed path from x to y .

Valid: **c**i**b**f**m**d**g**e**a**j**h**k

Valid: **b****c****d****f****i****h****g****j****m****k****e****a**

Invalid: **c****b****f****m****g****h****j****k****i****e****a**

Big question: Given a DAG, how can we perform a topological sort efficiently?



BEFORE YOU LEAVE

<https://forms.gle/U7LqCkPVkLWBAmfP8>

